L2: Introduction to the Relational Model

Md. Tariquzzaman
Junior Lecturer, Dept. of Computer Science and Engineering
Islamic University of Technology (IUT)

CSE 4307: Database Management Systems

Contents

2.1 Structure of Relational Databases
2.2 Database Schema
2.3 Keys
2.4 Schema Diagrams
2.5 Relational Query Languages
2.6 Relational Algebra
- 2.6.1 The Select Operation
- 2.6.2 The Project Operation
- 2.6.3 Composition of Relational Operations
- 2.6.4 The Cartesian-Product Operation
- 2.6.5 The Join Operation
- 2.6.6 Set Operations
- 2.6.7 The Assignment Operation
- 2.6.8 The Rename Operation
- 2.6.9 Equivalent Queries

2.1 Structure of Relational Databases

2.1 Structure of Relational Databases

A relational database consists of a collection of tables, each of which is assigned a unique name.

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Figure: The course relation.

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure: The instructor relation.

course_id	*prereq_id*
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

Figure: The prereq relation.

Example of an Instructor Relation

attributes
(or columns)

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

tuples
(or rows)

Relation Schema and Instance

- A1, A2, ..., An are attributes
- R = (A1, A2, ..., An) is a relation schema
- Example: instructor = (ID, name, dept_name, salary)
- A relation instance r defined over schema R is denoted by r (R).
- The current values a relation are specified by a table
- An element t of relation r is called a tuple and is represented by a row in a table

Relation Schema vs. Instance

Analogy: Think of a **blueprint** vs. an **actual building**.

- **Schema** → design or structure of the table.
- **Instance** → actual data present at a moment.

Example

```
instructor = (ID, name, dept_name, salary)
```

Relation Schema = Template

- Defines what **columns** exist.
- Like a form with blank fields.
- Structure remains same over time.

Example:

```
instructor = (ID, name, dept_name, salary)
```

Relation Instance = Actual Data

- The **filled form** with real values.
- Changes as rows are added or removed.

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000

Attributes

- The set of allowed values for each attribute is called the domain of the attribute
- Attribute values are (normally) required to be atomic; that is, indivisible
- The special value null is a member of every domain. Indicated that the value is “unknown”
- The null value causes complications in the definition of many operations

Attributes and Domains

- Each column = an **attribute**.
- Each attribute has a **domain** of allowed values.

Example:

- `salary` $\rightarrow$ valid numbers (0–1,000,000)
- `name` $\rightarrow$ valid text strings

Atomic and Null Values

Atomic Values

- Must be **indivisible**.
- `phone = "555-1234"`
- `phones = {"555-1234", "555-5678"}`

Null Values

- Means “unknown” or “not applicable”.
- Example: `phone = null`

Relations are Unordered

- Row order does **not matter!**
- These tables are identical:

Table A

ID	name
10101	Alice
12121	Bob

Table B

ID	name
12121	Bob
10101	Alice

Analogy Table

Concept	Analogy
Schema	Recipe card (design)
Instance	Actual ingredients today
Attribute	Ingredient slot (e.g., sugar)
Domain	Allowed range (0–5 cups)
Tuple	One complete recipe
Atomic	Single value only
Null	Ingredient missing

2.2 Database Schema

Database Schema

- Database schema – is the logical structure of the database.
- Database instance – is a snapshot of the data in the database at a given instant in time

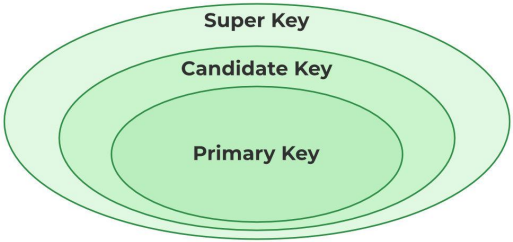
  - schema: *instructor* (*ID, name, dept_name, salary*)
  - Instance:

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

2.3 Keys

2.3 Keys — Overview

- **Super Key:** Any set of attributes that uniquely identifies a tuple (may include extra attributes).
- **Candidate Key:** A minimal super key — no extra attributes.
- **Primary Key:** One selected candidate key used as the official identifier (no NULLs).
- **Alternate Key:** Candidate keys not chosen as the primary key.



Super Key

Definition: A super key is any single attribute or combination of attributes that uniquely identifies a row. It may contain extra attributes unnecessary for uniqueness.

Facts:

- Can contain NULL values.
- Can include extra attributes.
- All candidate keys and primary keys are superkeys.

STUD_NO	SNAME	ADDRESS	PHONE
1	Shyam	Delhi	123456789
2	Rakesh	Kolkata	223365796
3	Suraj	Delhi	175468965

Table: STUDENT Table

Superkey examples: {STUD_NO}, {PHONE}, {STUD_NO, SNAME}

Candidate Key

Definition: A candidate key is a **minimal superkey**. It uniquely identifies each tuple and contains no unnecessary attributes.

Properties:

- Must contain unique values.
- Cannot contain duplicate records.
- A table may have multiple candidate keys.

STUD_NO	SNAME	ADDRESS	PHONE
1	Shyam	Delhi	123456789
2	Rakesh	Kolkata	223365796
3	Suraj	Delhi	175468965

Table: STUDENT Table

Candidate key examples: {STUD_NO}, {PHONE}

Composite Candidate Key

Definition: A candidate key that consists of **two or more** attributes.

STUD_NO	TEACHER_NO	COURSE_NO
1	001	C001
2	056	C005

Table: STUDENT_COURSE Table

Composite Candidate Key Example:

$$\{STUD_NO, COURSE_NO\}$$

Primary Key

Definition: A primary key is the candidate key chosen as the official unique identifier.

Properties:

- Must be unique.
- Cannot be NULL.
- Tables may have multiple candidate keys but only one primary key.
- Can be composite.

STUD_NO	SNAME	ADDRESS	PHONE
1	Shyam	Delhi	123456789
2	Rakesh	Kolkata	223365796
3	Suraj	Delhi	175468965

Table: STUDENT Table

Chosen Primary Key: {STUD_NO}

Alternate Key

Definition: Any candidate key that is **not chosen** as the primary key.
Facts:

- Still uniquely identifies records.
- May be single or multi-attribute.
- Often used for secondary indexing.

STUD_NO	SNAME	ADDRESS	PHONE
1	Shyam	Delhi	123456789
2	Rakesh	Kolkata	223365796
3	Suraj	Delhi	175468965

Table: STUDENT Table

If **STUD_NO** is chosen as the primary key, then **PHONE** becomes the **alternate key**.

Foreign Key

Definition: A **foreign key** in one table refers to the **primary key** of another table, creating a parent–child relationship and enforcing referential integrity.

Parent Table: STUDENT

STUD_NO	SNAME	ADDRESS	PHONE
1	Shyam	Delhi	123456789
2	Rakesh	Kolkata	223365796
3	Suraj	Delhi	175468965

Primary Key: STUD_NO

Child Table: ENROLLMENT

ENROLL_ID	STUD_NO	COURSE_NO
501	1	C001
502	2	C005
503	1	C003

Foreign Key: STUD_NO → STUDENT(STUD_NO)

Meaning: Every value of STUD_NO in `ENROLLMENT` must appear in the `STUDENT` table.

Foreign Key — Analogy

Imagine a **teacher's record book** that lists each student's *roll number*. That roll number must exist in the **main student register**.

- The teacher's record = *referencing relation*.
- The student register = *referenced relation*.

Moral: A foreign key ensures *what you refer to actually exists*.

Keys in Relational Model

- Let $K \subseteq R$
- K is a **superkey** of relation R if values of K are sufficient to uniquely identify each tuple of any $r(R)$.
- **Example:** $\{ID\}$ and $\{ID, name\}$ are both superkeys of `instructor`.
- Superkey K is a **candidate key** if K is *minimal*.
- **Example:** $\{ID\}$ is a candidate key for `instructor`.
- One of the candidate keys is chosen as the **primary key**.
- **Foreign key constraint:** Value in one relation must appear in another.
  - Referencing relation
  - Referenced relation
  - Example: `dept_name` in `instructor` referencing `department`.

2.4 Schema Diagrams

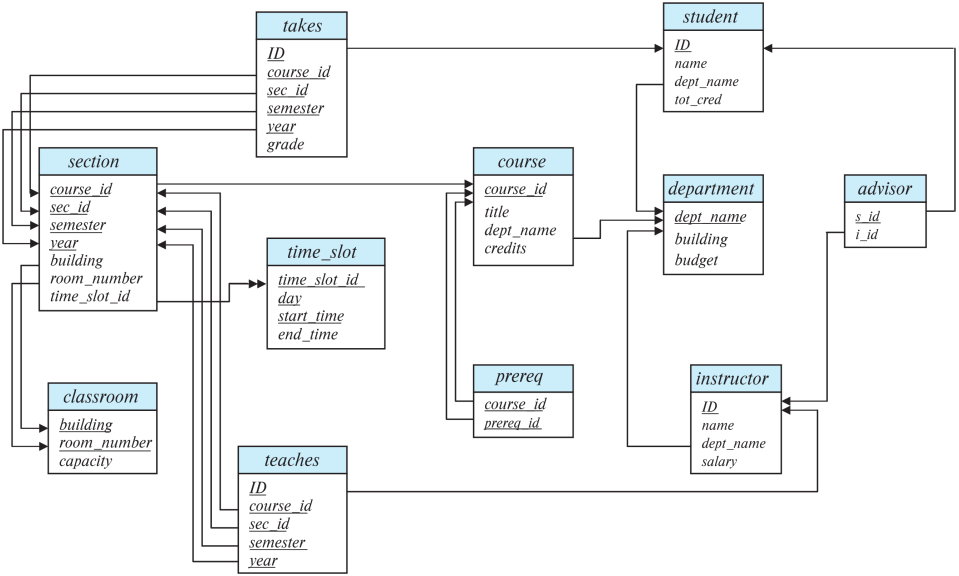
2.4 Schema Diagram — Analogy

Think of a **city map**.

- Each **building** = a **relation**.
- **Roads/arrows** = connections via foreign keys.
- **Underlined names** = main building identifiers (primary keys).

Moral: A schema diagram is the map of how data “buildings” connect.

Schema Diagram for a University Database



2.5 Relational Query Languages

Relational Query Languages

- Procedural vs Non-procedural (Declarative)
- **Pure languages:**
  - Relational algebra
  - Tuple relational calculus
  - Domain relational calculus
- These languages are equivalent in computing power
- We focus on **relational algebra** (not Turing-complete)
- Consists of 6 basic operations

Relational Query Languages

- **Analogy:** Think of a chef in a kitchen
  - **Procedural:** Chef follows a recipe step by step
  - **Non-procedural (Declarative):** Chef is told the dish to make, decides steps themselves
  - **Relational algebra / calculus:** Tools in the kitchen (knife, pan, blender) — different tools, same result

2.6 Relational Algebra

2.6 Relational Algebra — Analogy

A **toolbox of data operations**.

- Each tool (select, project, join, etc.) takes tables and builds new ones.
- Just like LEGO pieces, their outputs can connect to the next operation.
- A procedural language consisting of a set of operations that take one or two relations as input and produce a new relation as their result.

You build complex queries step by step from smaller blocks.

Relational Algebra: Basic Operators

Six basic operators:

- **Select:** σ
- **Project:** Π
- **Union:** $\cup$
- **Set difference:** $-$
- **Cartesian product:** $\times$
- **Rename:** ρ

2.6.1 Select — Analogy

Think of a **filter in a spreadsheet**. Only rows matching a condition stay visible.
Select = pick only what satisfies your condition.

Select Operation

- Selects tuples that satisfy a predicate p:

$$\sigma_p(r)$$

- Example: Instructors in `Physics`:

$$\sigma_{\text{dept_name = "Physics"}}(\text{instructor})$$

- Here, predicate, p = dept_name - "Physics", r is the relation (table) = instructor

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

Figure: Query Result

Select Operation Cntd.

- Predicates may use $=, \neq, >, \geq, <, \leq$
- Combine predicates with $\wedge$ (and), $\vee$ (or), $\neg$ (not)
- Example: Physics instructors with salary $> 90,000$:

$$\sigma_{\text{dept_name = "Physics"} \wedge \text{salary > 90000}}(\text{instructor})$$

- Comparisons can involve attributes:

$$\sigma_{\text{dept_name = building}}(\text{department})$$

2.6.2 Project — Analogy

Imagine you **hide columns** in Excel to see only names and salaries.
Project = choose specific attributes to show.

Project Operation

- Unary operation that returns relation with selected attributes.
- Notation:

$$\Pi_{A_1, A_2, \ldots, A_k}(r)$$

- where A1, A2, ..., Ak are attribute names and r is a relation name.
- Removes duplicate rows (since relations are sets)
- The result is defined as the relation of k columns obtained by erasing the columns that are not listed

Project Operation Example

- Example: eliminate `dept_name` from instructor:

$$\Pi_{ID,name,salary}(\text{instructor})$$

ID	*name*	*salary*
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

Figure: Query Result

2.6.3 Composition — Analogy

Like solving a puzzle by connecting small pieces: output of one operation becomes input to the next.
Operations can be chained like steps in a formula.

- Relational-algebra operations return relations, enabling composition.
- Example: Find names of all instructors in `Physics`:

$$\Pi_{\text{name}}(\sigma_{\text{dept_name} = \text{"Physics"}}(\text{instructor}))$$

2.6.4 Cartesian Product — Analogy

Imagine pairing every **shirt** with every **pant** to see all outfit combos.
Cartesian product = all possible pairs of rows.

- Denoted by $\times$, combines tuples from two relations.
- Example:

$$\text{instructor} \times \text{teaches}$$

- Construct tuples for every pair (t_i, t_j) where $t_i \in \text{instructor}, t_j \in \text{teaches}$
- Distinguish same-named attributes:

$$\text{instructor.ID}, \text{teaches.ID}$$

2.6.4 Cartesian Product — Example Tables

ID	Name
I1	Smith
I2	Lee

Table: Instructor

CourseID	Semester
C1	Fall
C2	Spring

Table: Teaches

ID	Name	CourseID	Semester
I1	Smith	C1	Fall
I1	Smith	C2	Spring
I2	Lee	C1	Fall
I2	Lee	C2	Spring

Table: Instructor × Teaches

Each instructor is paired with every teaching record — producing all possible combinations.

2.6.4 Distinguishing Attributes with Same Name

When two relations share attributes with the same name (e.g., `ID`), the Cartesian product creates ambiguity.

- Each relation may have an attribute `ID`.
- To distinguish them, prefix with the relation name:

$$\text{instructor.ID}, \quad \text{teaches.ID}$$

- Ensures clarity in combined results or joins.

instructor.ID	Name	teaches.ID	CourseID
I1	Smith	I1	C1
I1	Smith	I1	C2
I2	Lee	I2	C1

Now, it's clear which `ID` comes from which relation.

2.6.5 Join — Analogy

The **matchmaker** of databases: joins two tables by matching a shared key (like instructor ID).
Join = combine only related pairs, not everyone.

- Combines select and Cartesian product:

$$r \bowtie_\theta s = \sigma_\theta(r \times s)$$

- Example:

$$\text{instructor} \bowtie_{\text{instructor.ID = teaches.ID}} \text{teaches}$$

2.6.5 The Join Operation

- The Cartesian-Product associates every tuple of *instructor* with every tuple of *teaches*.

$$instructor \times teaches$$

  - Most of the resulting rows have information about instructors who did **NOT** teach a particular course.
- To get only those tuples of "*instructor* $\times$ *teaches*" that pertain to instructors and the courses that they taught, we write:

$$\sigma_{instructor.id=teaches.id}(instructor \times teaches)$$

  - We get only those tuples of "*instructor* $\times$ *teaches*" that pertain to instructors and the courses that they taught.
- The result of this expression, shown in the next slide.

2.6.6 Set Operations — Analogy

Like combining or comparing **student lists** from two classes:

- **Union:** all students from both classes.
- **Intersection:** students in both.
- **Difference:** students in one but not the other.
- Union: $r \cup s$ (compatible relations, same arity)
- Intersection: $r \cap s$
- Set difference: $r - s$

Treat tables like sets for logical combinations.

Set Operations (Union)

For $r \cup s$ to be valid.

- r, s must have the same arity (same number of attributes)
- The attribute domains must be compatible (example: 2nd column of r deals with the same type of values as does the 2nd column of s)
- Example: to find all courses taught in the Fall 2017 semester, or in the Spring 2018 semester, or in both

$$\Pi_{\text{course_id}}(\sigma_{\text{semester="Fall", year=2017}}(section)) \cup$$

$$\Pi_{\text{course_id}}(\sigma_{\text{semester="Spring", year=2018}}(section))$$

Set Operations (Intersection)

The set-intersection operation allows us to find tuples that are in both the input relations. For $r \cap s$ to be valid.

- r, s have the same arity
- attributes of r and s are compatible
- Example: Find the set of all courses taught in both the Fall 2017 and the Spring 2018 semesters

$$\Pi_{\text{course_id}}(\sigma_{\text{semester="Fall", year=2017}}(section)) \cap$$

$$\Pi_{\text{course_id}}(\sigma_{\text{semester="Spring", year=2018}}(section))$$

Set Operations (Difference)

The set-difference operation allows us to find tuples that arein one relation but are not in another.

- r, s have the same arity
- attributes of r and s are compatible
- Example: Find all courses taught in the Fall 2017 semester, but not in the Spring 2018 semester

$$\Pi_{\text{course_id}}(\sigma_{\text{semester="Fall", year=2017}}(section)) -$$

$$\Pi_{\text{course_id}}(\sigma_{\text{semester="Spring", year=2018}}(section))$$

2.6.7 Assignment — Analogy

Think of using **temporary notes** in math: You compute one result, store it as a variable, then reuse it. **Assignment** = store intermediate results for clarity.

2.6.7 Assignment Operation

It is convenient to write a relational-algebra expression by assigning parts of it to temporary relation variables.

- The assignment operation is denoted by $\leftarrow$ (or sometimes $\neg$).
- Works like assignment in a programming language.

Example: Find all instructors in the *Physics* and *Music* departments.

$$\text{Physics} \leftarrow \sigma_{\text{dept_name}=\text{"}Physics\text{"}}(\text{instructor})$$
$$\text{Music} \leftarrow \sigma_{\text{dept_name}=\text{"}Music\text{"}}(\text{instructor})$$
$$\text{Result} = \text{Physics} \cup \text{Music}$$

With the assignment operation, a query can be written as a sequential program of assignments followed by an expression whose value is displayed as the result.

2.6.8 Rename — Analogy

Sometimes two relations or attributes have the same name. To avoid confusion, we give them temporary unique names — just like giving nicknames to people with the same name.

- **Assignment:** Store the result of a query in a temporary relation

$$T \leftarrow \sigma_{\text{dept_name = "Physics"}}(\text{instructor})$$

  Here, `T` is the temporary relation storing instructors from Physics.

- **Rename:** Give a new name to a relation or its attributes

$$\rho_x(E) \quad \text{or} \quad \rho_{x(A_1, A_2, \ldots, A_n)}(E)$$

  Here, `x` is the new name for the relation, and optionally the attributes are renamed too.

Analogy: Assignment is like saving a list in a box named 'T'; rename is like putting a label on that box or renaming items inside it.

2.6.8 Rename Operation

The results of relational-algebra expressions do not automatically have a name that we can use to refer to them. The **rename operator** (ρ) provides a way to name results.

- $\rho_x(E)$: returns the result of expression E under the name x
- $\rho_x(A_1, A_2, \ldots, A_n)(E)$: renames the attributes as well

Example:

$$\rho_{\text{T1}}(\sigma_{\text{salary}>90000}(\text{instructor}))$$

This expression names the resulting relation as *T1*.

2.6.9 Equivalent Queries — Analogy

Just like **different routes** lead to the same destination, multiple relational algebra expressions can yield the same result. *Many paths, one answer.*

- Multiple relational-algebra expressions can be equivalent.
- Example: Physics instructors with salary $> 90,000$

$$\sigma_{\text{dept_name = "Physics"} \wedge \text{salary > 90000}}(\text{instructor}) \equiv$$

$$\sigma_{\text{dept_name = "Physics"}}(\sigma_{\text{salary > 90000}}(\text{instructor}))$$

Questions?